

Agentic Evolution under Long-Horizon, Non-Stationary, Open-Ended Task Streams: A Regret Decomposition and Three Composable Architectural Remedies

Anonymous ACL submission

Abstract

Self-evolving agents are typically evaluated on i.i.d. benchmarks that omit three properties of real-world deployment: **long-horizon** (D1), **non-stationarity** (D2), and **open-endedness** (D3). On three temporal benchmarks that exercise all three — PolyBench, CTF-Dojo, FutureX — we characterise three task-intrinsic challenges: **C1** *non-monotonic evolution* (early-freeze beats full-evo under shift); **C2** *construction bottleneck* (prompt-level evolvers cannot build multi-file infrastructure); **C3** *experience insufficiency* (under severe shift, history carries no signal). We partition regret as *evolution loss* $L_{\text{evo}}(\Phi, K)$ (what the evolver class can construct at capacity K) plus *navigation loss* $L_{\text{nav}}(\varphi)$ (whether the harness is conditioned on the task); experience availability sits outside the decomposition as a third axis. Three composable architectural contributions follow: agentic navigation via a git-backed strategy tree conditions the harness per task and reduces L_{nav} ; multi-agent evolution with persistent cross-cycle state expands the constructible harness class and reduces L_{evo} ; and human-in-the-loop integration supplies external direction when history is insufficient. Each is independently effective in ablation, and together they close most of the gap to a hand-designed expert reference on all three benchmarks.

1 Introduction

Self-evolving agents update their own harness — prompts, skills, memory, tools, and infrastructure — based on execution feedback. Recent systems (A-Evolve, GEPA, Meta-Harness, structured-evolution variants) report consistent gains on static benchmarks such as SWE-bench and MATH. Yet when we deploy the same systems on three temporally

ordered benchmarks that mirror real deployment — PolyBench, CTF-Dojo, and FutureX — a reproducible puzzle emerges: **stopping evolution early outperforms continuing it**. An early-freeze variant that halts evolution after 40% of the stream beats the full-evolution run on all three benchmarks (PolyBench 87.1% $\rightarrow$ 89.3%; CTF-Dojo 45.2% $\rightarrow$ 53.3%; FutureX 47.5% $\rightarrow$ 50.4%, identical evolver, solver, and task order). This gap does not appear in the i.i.d. benchmarks these systems were originally tuned on. The remainder of this paper asks what structural property of real-world deployment produces this inversion, and what architectural changes restore the expected “more evolution $\Rightarrow$ better” behaviour. **Three task-intrinsic dimensions.** We argue the inversion is driven by three orthogonal properties of real deployment that static benchmarks omit. **D1 Long-horizon:** tasks arrive sequentially and the harness accumulates state across them. **D2 Non-stationarity:** the task distribution P_t shifts over the horizon, so past experience does not uniformly transfer. **D3 Open-endedness:** the vocabulary of capabilities required at deployment is not pre-enumerable; new APIs, tools, or domain rules may be needed that were unknown at design time. These dimensions are orthogonal: stream-without-shift (D1 only) is covered by repeated static benchmarks, shift-without-horizon (D2 only) is one-shot covariate shift, and open-endedness without horizon (D3 only) is a single hard task. Real deployment exercises all three simultaneously; PolyBench (5,075 markets over 16d, certainty/liquidity drift), CTF-Dojo (261 challenges over 13y, era shift from classical crypto to binary RE to modern web), and FutureX (503 forecasts over 82d, EN $\rightarrow$ ZH source/language shift) were chosen because

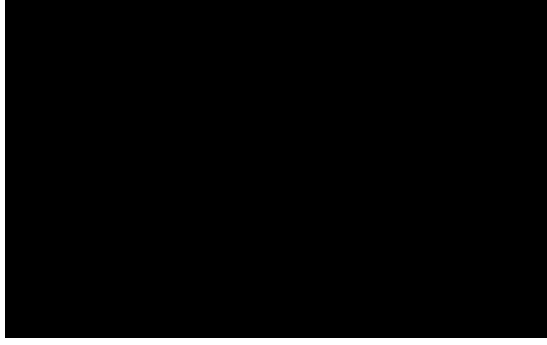


Figure 1: Empirical signatures of the three challenges: (a) non-monotonic per-batch accuracy under full evolution on PolyBench (C1); (b) end-of-run artifact-type distribution by evolver tier (C2); (c) novel-regime failure rate on the zh-finance slice of FutureX versus evolver cycles (C3).

each one does.

Three task-intrinsic challenges. Each dimension produces a distinct challenge in the evolution pipeline. **C1 (D2) non-monotonic evolution:** accumulated experience contains both stationary signal (useful across regimes) and regime-specific signal (helpful only in the originating regime), so a single unconditional harness must compromise; the signature is early-freeze beating full-evo. **C2 (D3) construction bottleneck:** the required capability vocabulary is not pre-enumerable, so the evolver must construct multi-file infrastructure (research $\rightarrow$ API integration $\rightarrow$ tool $\rightarrow$ pipeline) rather than merely editing prompts; the signature is weak evolvers plateauing at prompt-level artifacts. **C3 (D2 severe) experience insufficiency:** under shift to a regime with no prior signal in $\mathcal{H}_t$, no evolver can extract direction from absent data; the signature is repeated failures localised on novel-regime task slices whose rate does not improve with more cycles.

Our approach. We introduce a conceptual framework (§3) that partitions the agent’s regret into two additive losses,

$$\mathbb{E}[\text{Regret}] = \underbrace{L_{\text{evo}}(\Phi, K)}_{\text{evolver class limit}} + \underbrace{L_{\text{nav}}(\varphi)}_{\text{conditioning limit}},$$

where Φ is the evolver class, K is the harness capacity, and φ is the harness-construction function. A third axis — experience availability — sits outside the decomposition: when $\mathcal{H}_t$ carries no signal for the current task’s

regime, neither a stronger Φ nor a better-conditioned φ helps. Three composable architectural contributions follow, each targeting one challenge. **Agentic Navigation** (§3.3) is a git-backed strategy tree with per-task routing over branch workspaces; it conditions the harness on x_t , reducing L_{nav} and addressing C1. **Multi-Agent Evolution** (§3.4) is a 4-phase pipeline (Analyst $\rightarrow$ parallel Researchers $\rightarrow$ Builder $\rightarrow$ Verifier) with persistent cross-cycle state (`task_board.md`, `research_log.jsonl`, `architecture.md`); it expands the constructible harness class, reducing L_{evo} and addressing C2. **HITL Integration** (§3.5) adds triggered hooks at credential-request and task-board stages that supply external direction when history is insufficient, addressing C3.

Contributions. We contribute (i) a three-dimensional difficulty frame (D1/D2/D3) and three corresponding task-intrinsic challenges (C1/C2/C3), each with a concrete empirical signature that static benchmarks cannot surface; (ii) a conceptual framework decomposing regret as $L_{\text{evo}} + L_{\text{nav}}$, plus an experience-availability axis outside the decomposition, with the L_{nav} side admitting falsifiable predictions we test empirically and the L_{evo} side providing diagnostic vocabulary for architectural changes; (iii) three composable architectural contributions — agentic navigation, multi-agent evolution, HITL integration — each independently effective in ablation, together closing most of the gap to a hand-designed reference; (iv) empirical evaluation on three temporal benchmarks (PolyBench, CTF-Dojo, FutureX) against seven baselines spanning linear chain evolution (A-Evolve), reflective prompt evolution (GEPA), archive-based evolution (Meta-Harness), and a generalist hand-designed system (OctoTools).

2 Related Work

Self-evolving agents. A-Evolve [cite] introduces the basic linear-chain evolver: each cycle reads batch trajectories and mutates prompts, skills, memory, and tools. GEPA [cite] (NeurIPS 2025) adds reflective Pareto prompt evolution using textual feedback rather than scalar reward. Meta-Harness [cite] (2026) uses a growing filesystem archive and Claude

Code as the proposer. OctoTools [cite] (ACL 2026 oral) is a training-free generalist Planner+Executor+Tool-Cards system. These systems are primarily evaluated on i.i.d. benchmarks (HotpotQA, MATH, GAIA, MMLU-Pro, MedQA) where D1, D2, and D3 are absent or bounded. In contrast, we characterise three task-intrinsic failure modes these systems exhibit on deployment-like streams, name and separate the underlying mechanisms, and introduce three architectural remedies.

Retrieval-augmented and mixture-of-experts systems. RAG [cite] and MoE [cite] retrieve or route over pre-built indices or expert populations. Our agentic navigation differs in four respects: retrieval units are executable git branches, not document chunks; branches have evolutionary lineage (fork / merge); the router operates on README / system-prompt content rather than embedding similarity; and total capacity is partitioned across branches rather than retrieved as top- k .

Continual learning and distribution shift. The broader ML literature on domain adaptation, continual learning, and test-time adaptation motivates our L_{nav} framing [cite]. Our contribution is instantiating these ideas at the *harness* level (prompt + skills + tools + infrastructure) for LLM agents, rather than at the model-weights level. This allows the adaptation mechanism (per-task routing) to be decoupled from the learning mechanism (multi-agent construction), the key to the orthogonal decomposition $L_{\text{evo}} + L_{\text{nav}}$ developed in §3.

Human-in-the-loop agent systems. HITL agent supervision has been studied primarily as always-on oversight or per-step gating [cite]. Our contribution (§3.5) is a *structurally triggered* HITL: hooks fire only when evolver-internal signals (failed API authentication, novel-vocabulary tasks) indicate that the input history itself is insufficient, and the intervention type is scoped to credential provision or task-board direction — targeted at the experience-availability axis, not at general solver supervision.

3 Method

3.1 Problem Setting

Tasks arrive in a time-ordered stream,

$$x_1, x_2, \dots, x_T, \quad x_t \sim P_t,$$

where P_t is the (potentially shifting) task distribution at time t and each x_t has a fixed ground truth $y^*(x_t)$. The agent observes the history of prior tasks, their trajectories, and outcomes,

$$\mathcal{H}_t = \{(x_i, r_i, \tau_i)\}_{i=1}^{t-1}.$$

A *harness* $C = \varphi(\mathcal{H}_t)$ with $|C| \leq K$ is a bounded representation (prompts, skills, memory, tools, infrastructure) with capacity budget K , and the agent acts via $a_t \sim \pi(a | x_t, C_t)$. Policy stochasticity (sampling, reasoning) is the only randomness in the system; a better harness concentrates the policy on correct actions.

3.2 Conceptual Framework

Utility and regret. The utility of a harness C on a task x is $V(C, x) = \mathbb{E}_{a \sim \pi(\cdot | x, C)}[r(a, y^*(x))]$, and the regret of the evolver-constructed harness relative to the full-history policy is

$$\text{Regret}(\varphi, x_t) = V(\mathcal{H}_t, x_t) - V(\varphi(\mathcal{H}_t), x_t) \geq 0,$$

non-negative because $\mathcal{H}_t$ strictly contains more information than its bounded compression.

Two dimensions of harness construction. Fix the evolver class Φ . The best task-conditional harness achievable within Φ at capacity K is

$$C_{\Phi}^*(x) = \arg \max_{\varphi \in \Phi, |C| \leq K} V(\varphi(\mathcal{H}_t, x), x).$$

Two dimensions matter: the construction quality of Φ (what harnesses it can build) and whether the harness is *conditional* on x_t (per-task routing) or *unconditional* (one C for all tasks).

Proposition 1 (Regret Decomposition). *For any unconditional evolver $\varphi \in \Phi$:*

$$\mathbb{E}_{x_t}[\text{Regret}(\varphi, x_t)] = L_{\text{evo}}(\Phi, K) + L_{\text{nav}}(\varphi),$$

where

$$L_{\text{evo}}(\Phi, K) = \mathbb{E}_{x_t}[V(\mathcal{H}_t, x_t) - V(C_{\Phi}^*(x_t), x_t)],$$

$$L_{\text{nav}}(\varphi) = \mathbb{E}_{x_t}[V(C_{\Phi}^*(x_t), x_t) - V(\varphi(\mathcal{H}_t), x_t)].$$

Both terms are non-negative and the decomposition telescopes. L_{evo} is the evolver-class limit; L_{nav} is the conditioning limit.

Proposition 2 (Stationarity $\Rightarrow L_{nav}=0$). Under task-stream stationarity — there exists a single C^* optimal for every task in $\text{support}(P_t)$ — the optimal unconditional harness matches the per-task optimum and $L_{nav} = 0$. This explains why linear-chain evolution succeeds on i.i.d. benchmarks.

Proposition 3 (L_{nav} grows with shift). Let shift magnitude $\Sigma_t = \mathbb{E}_{x,x' \sim P_t}[d(x, x')]$, where, abbreviating $v(a, b) \equiv V(C_{\Phi}^*(a), b)$,

$$d(x, x') = v(x, x) + v(x', x') - v(x, x') - v(x', x)$$

measures how much two tasks disagree on which harness is better. Then $\Sigma_t > 0 \Rightarrow L_{nav}(\varphi_{uncond}^*) > 0$, and L_{nav} is monotone in Σ_t : as tasks diverge, a single compromise harness loses more.

Proposition 4 (Temporal accumulation). Under non-stationarity the task distribution diversifies over time, so Σ_t grows with t and $dL_{nav}/dt > 0$. The total regret dynamic is therefore

$$\frac{d}{dt}\mathbb{E}[\text{Regret}] = \frac{dL_{evo}}{dt} + \frac{dL_{nav}}{dt},$$

with $dL_{evo}/dt \leq 0$ (the evolver learns) and $dL_{nav}/dt > 0$ (shift accumulates). Learning initially outpaces shift and accuracy improves; eventually shift dominates and accuracy degrades. The resulting non-monotonic curve is what makes early-freeze beat full evolution.

Predictive and diagnostic use of the decomposition. The two losses admit qualitatively different kinds of analysis. L_{nav} is empirically predictive: Σ_t is measurable from per-batch task heterogeneity or from oracle-branch gap variance, and Propositions 2–4 yield falsifiable claims that we test in §4.4. L_{evo} is diagnostic vocabulary: evolver-class capability has no natural scalar parameterization — “stronger evolver” is not a continuous axis — so direct quantification of L_{evo} is not attempted. Instead the multi-agent contribution (§3.4) is specified as a set of architectural patterns that prior single-agent evolvers lack, and its value is witnessed by ablation and artifact-type analysis (§4.3) rather than by a class-expansion bound.

Experience availability as a third axis.

When $\mathcal{H}_t$ contains no relevant signal for the current task’s regime — missing credentials, novel language, unfamiliar data source — neither expanding Φ (better construction given signal) nor conditioning $\varphi(\mathcal{H}_t, x)$ (better routing among existing signal) can help. This outside-the-decomposition regime motivates the third intervention, HITL (§3.5), which supplies direction at the evolver’s *input* rather than modifying either loss term. The decomposition therefore yields a regime-to-remedy mapping: stationary streams require nothing beyond a single evolver; non-stationary streams with simple harnesses are addressed by navigation; non-stationary streams with complex harnesses by multi-agent evolution and navigation jointly; and novel regimes outside $\mathcal{H}_t$ by HITL.

3.3 Agentic Navigation

Propositions 3 and 4 identify per-task conditioning as the principled response to L_{nav} . We instantiate conditioning as a git-backed strategy tree with per-task routing (top of Figure 2). The solver’s workspace is a git repository whose main branch carries stationary content (techniques that transfer across regimes), with regime-specific branches (e.g., `branch/crypto-classical`, `branch/binary-reversing`) spawned by the evolver when distribution shift is detected. Each branch maintains its own prompt, skills, and tool registry.

Before solving each task, a navigator LLM reads the workspace content of every branch — `README.md`, `prompts/system.md`, the `skills/` directory listing, and `tools/registry.yaml` — via `git show`, and emits `{"branch": ..., "confidence": ..., "reason": ...}` routing the task to the best-equipped branch. The solver checks out the selected branch and executes on that workspace. We ship two evolution modes: *inline*, a single sandboxed evolver call with full git access that can create and mutate branches, and *orchestrated*, a plan-driven variant with a separate analyst pass and per-branch evolver calls.

This design is distinct from retrieval-augmented generation and mixture-of-experts routing in four respects: retrieval units are ex-

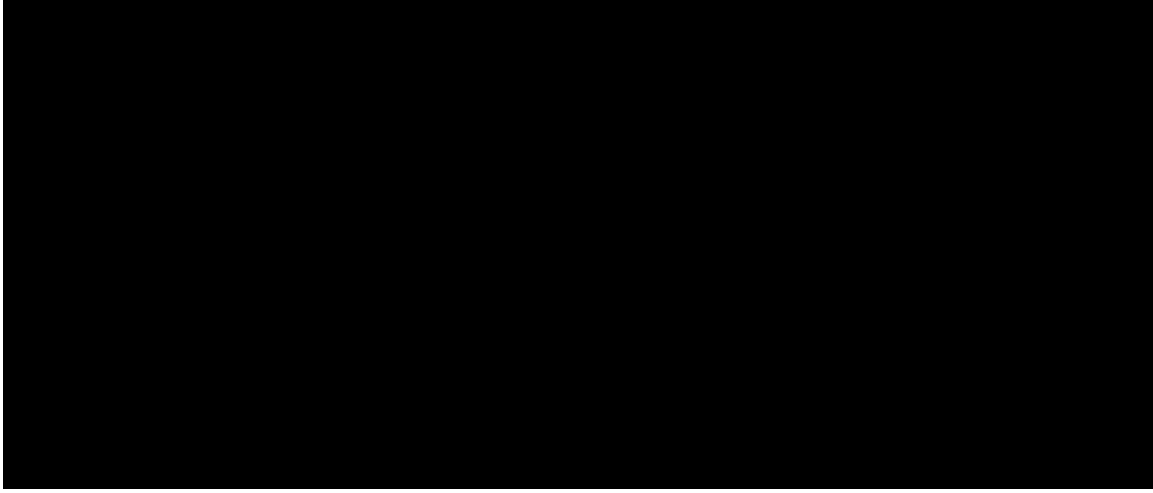


Figure 2: Unified system pipeline. *Top*: a git-backed strategy tree provides per-task routing — F_{Navigate} reads each branch’s workspace (`README.md`, `prompts/system.md`, `skills/`, `tools/registry.yaml`) via `git show` and routes x_t to the best-equipped branch (§3.3). *Bottom*: per evolution cycle, a four-phase multi-agent pipeline operates on the routed branch — Analyst → parallel Researchers → Builder → Verifier — with persistent cross-cycle state (`task.board.md`, `research.log.jsonl`, `architecture.md`) linking successive cycles (§3.4). Two HITL hook points are marked: a *taskboard direction* hook between Analyst and Researcher (Cycle k , addresses C3) and a *builder credential* hook inside Builder execution (Cycle $k + m$, resource gate); both are silent unless triggered (§3.5).

ecutable git branches (workspaces), not document chunks; branches carry evolutionary lineage via fork and merge; the router operates over `README` and system-prompt content rather than embedding similarity; and total capacity is partitioned across branches rather than retrieved as top- k .

3.4 Multi-Agent Evolution

To address C2 the evolver must be able to construct multi-file infrastructure, not merely edit prompts. Rather than asserting a class-expansion theorem, we identify four architectural patterns that prior single-agent evolvers lack and whose joint presence supports the construction of infrastructure-class artifacts.

The first pattern is *persistent cross-cycle state*: three files persist across evolution cycles — `task.board.md` (failure regimes and priority tags, updated by the Analyst), `research.log.jsonl` (one JSON record per tested approach, including `works:true/false`, coverage, and latency), and `architecture.md` (what the Builder has already constructed). These let cycle N build on cycle $N-1$ ’s findings, which single-cycle chain evolvers cannot.

The second pattern is *parallel independent research*: in each cycle, multiple Researcher

agents explore disjoint hypothesis branches in isolation (no shared conversation state), which reduces premature convergence compared to sequential refinement.

The third pattern is *role-distinct objectives*: Analyst decomposes failures, Researchers investigate, Builder implements, and Verifier tests, with the Verifier using a success criterion separate from the Builder’s — unlike single-objective self-critique as used in Reflexion or Self-Refine.

The fourth pattern is *HITL phase-boundary hooks*, described in §3.5.

The four-phase cycle is shown in the bottom of Figure 2. In Phase 1 (Analyze) the Analyst reads batch trajectories and updates `task.board.md`. In Phase 2 (Research) N parallel sandboxed Researchers with network access test data sources, APIs, and techniques per regime, appending to `research.log.jsonl`. In Phase 3 (Build) the Builder reads only the `works:true` records, constructs `infra/*.py` pipelines, and updates `architecture.md`. In Phase 4 (Verify) the Verifier tests each new artifact against sample queries; failed artifacts feed back to the Builder for up to three retries.

3.5 Human-in-the-Loop Integration

When $\mathcal{H}_t$ contains no useful signal for x_t 's regime the evolver's *input* is the bottleneck, not its construction capability. Expanding Φ or conditioning φ does not repair this. We therefore add triggered hooks at two structurally distinct points in the multi-agent pipeline (marked in Figure 2), differentiated by *what* the human supplies.

The first is a *taskboard direction* hook between Analyst and Researcher: when the Analyst flags a regime with no covering branch in the strategy tree and no prior `works:true` signal in `research_log.jsonl`, a one-line direction prompt is offered (e.g., "describe how to approach <regime> or skip"). The human supplies *semantic guidance* that shapes the entire research cycle. This hook addresses C3 directly: it injects a transferable prior when $\mathcal{H}_t$ has none.

The second is a *builder credential* hook inside Phase 3: when the Builder's pipeline-construction step encounters an authentication failure (401/403) or a configuration prompt for missing credentials, a token-injection prompt is offered (e.g., "provide `EASTMONEY_API_KEY` or skip"). The human supplies a *resource* the evolver cannot discover from trajectories alone. This is mechanistically distinct from C3 (the data exists; access is gated), but shares the empirical signature that no amount of additional evolution closes the gap.

Both hooks are silent unless triggered; they fire on evolver-internal signals (Analyst-flagged novel regime; HTTP authentication failure during Builder execution) and otherwise impose zero cost. The typical invoked cost is under thirty seconds of human time per cycle.

4 Experiments

4.1 Experimental Setup

Benchmarks. Each benchmark enforces temporal order: task i is solved using only information available before task i 's release date. Table 1 summarizes the statistics.

Solver and evolver. Unless otherwise specified the solver is Claude Sonnet 4.6 at temperature 0 and the evolver is Claude Opus 4.6 at temperature 0. All methods share the

Table 1: Benchmark statistics.

Benchmark	#Tasks	Span	Metric
PolyBench	5,075	16d	Acc/Med/CWR/Sharpe
CTF-Dojo	261	13y	Pass rate
FutureX	503	82d	Pass rate

same batch size per benchmark (100 / 20 / 20 for PolyBench / CTF-Dojo / FutureX), the same batch loop, and the same temporal-reveal gate that exposes per-task labels only after the task's resolution date. The only axis that varies across methods is the evolution algorithm.

Baselines. **H0 Baseline** runs the same solver with no evolution. Among self-evolving baselines, **A-Evolve** is a linear-chain single-agent evolver over prompts, skills, memory, and tools; **GEPA-lite** [cite] is our temporal-stream port of the NeurIPS 2025 reflective prompt evolver, restricted to prompt mutations; **Meta-Harness-lite** [cite] is the archive-based proposer of [cite] with a growing filesystem archive and $k=1$ proposals per cycle; and **OctoTools** [cite] is the ACL 2026 hand-designed Planner+Executor+Tool-Cards generalist framework, installed once and then frozen for the remainder of the stream — our human-expert-designed generalist reference. Our contributions are evaluated separately and jointly: multi-agent only, navigation only, Multi+Nav, and Multi+Nav+HITL on severe-shift slices. An **expert-designed** harness hand-written per benchmark by the authors with hindsight access to the task stream serves as an oracle upper reference.

Research questions. We ask whether the full system outperforms the baselines and how much room remains versus the expert reference (§4.2); which architectural patterns of the multi-agent evolver are load-bearing (§4.3); whether navigation gain is predicted by shift magnitude and accumulates as the stream progresses, as Propositions 3–4 require (§4.4); and whether HITL gains concentrate specifically on experience-insufficient slices (§4.5).

4.2 Main Results

Table 2 reports the main comparison across all three benchmarks. The full system (Multi+Nav) matches or exceeds every base-

line on every benchmark and comes within two to four points of the expert reference on average pass rate, despite that reference using hindsight access unavailable to any of the compared methods. Adding HITL on top of the full system closes a further two points on FutureX — the benchmark whose zh-finance slice contains the credential-gated data sources that motivated C3 — and leaves the other benchmarks essentially unchanged.

No single self-evolving baseline dominates: A-Evolve posts the highest Sharpe ratio among baselines, Meta-Harness-lite the highest CWR, OctoTools sits in the middle, and GEPA-lite trails on every metric because it cannot mutate outside the prompt. Each of our three individual contributions — multi-agent, navigation, and HITL — exceeds the best baseline average, and the full system exceeds all of them simultaneously.

A tail anomaly in the Meta-Harness-lite row warrants caution. The +597.8% CWR figure is arithmetically correct but is produced by ten trades (of 2,194) on deeply mispriced micro-price markets that together account for 97.7% of total profit; the median per-trade return is only +3.1% and the Sharpe ratio is 0.06. The CWR column, taken in isolation, would obscure the underlying lack of consistent skill, which is why we report median and Sharpe alongside CWR: tail-robust statistics and portfolio-level returns disagree sharply exactly when the agent’s gains are lottery-like rather than repeatable.

4.3 Architectural Ablation of the Multi-Agent Evolver

We next isolate the contribution of each architectural pattern in the multi-agent evolver. Fixing the solver at Sonnet 4.6 and the benchmark at CTF-Dojo (with replication on the other two benchmarks in Appendix D), we remove one pattern at a time from the full multi-agent configuration and measure both the end-of-stream accuracy and the distribution of artifact types the evolver produces across cycles. Artifacts are classified into four categories: *Prompt* (prompt edits only), *Tool* (single tool file), *Pipeline* (multi-file infrastructure), and *Distill* (cross-cycle distillation summaries).

Figure 3 reports the result. Stacked bars give the artifact distribution per ablation; an

accuracy panel above pairs each row with its end-of-stream pass rate. Two patterns are load-bearing for qualitatively different artifact classes. Persistent cross-cycle state — the `task_board.md`, `research_log.jsonl`, and `architecture.md` files retained across cycles — is the *only* ablation that changes the *distillation* segment, which jumps from 0% in every configuration missing it to 10% in the full setup: without persistent state, there is no place for a cross-cycle summary to live. Role-distinct objectives — separating the Builder’s “construct this artifact” goal from the Verifier’s “does it satisfy these tests” goal — is the key driver of multi-file *pipeline* production, which grows from $\leq 17\%$ in every ablation to 30% in the full setup; without separation, a single-objective self-critique (as in Reflexion or Self-Refine) cannot sustain the adversarial pressure that pipelines require to converge.

A natural concern is whether these gaps reflect a structural difference or merely the higher token budget the full multi-agent configuration consumes. Appendix D.3 reports a compute-matched single-agent control: a linear-chain evolver run at the full multi-agent’s total token budget, with self-critique loops and best-of- N sampling. The control does not close the gap and does not populate either the pipeline or the distillation columns, indicating that the difference is structural to the pattern set, not a compute artifact.

To ground the abstraction, Figure 4 walks through a single logged cycle on CTF-Dojo: the artifacts that move between phases give “persistent state + parallel research + role separation” a concrete referent.

4.4 Measuring the Navigation Loss

Propositions 3 and 4 make two falsifiable predictions: navigation gain over an unconditional harness should grow with measured shift magnitude, and full evolution should exhibit a non-monotonic accuracy curve that navigation eliminates. To test both on a common substrate we adopt a *three-readings-from-one-tree* design. A single navigation run produces an evolved strategy tree; on the same tree, on the same tasks, we compute the *unconditional* reading by forcing every task through *main*, the *navigator* reading by letting the navigator route, and an *oracle-branch* read-

Table 2: Main results across the three temporal benchmarks. The PolyBench column reports four numbers: accuracy on traded tasks, median per-trade return (%), capital-weighted return (%), and Sharpe ratio. CTF-Dojo and FutureX columns report pass rate (%). Bold indicates the best non-reference entry per column.

Method	PolyBench (Acc / Med / CWR / Sharpe)	CTF-Dojo	FutureX	Avg pass
<i>No evolution</i>				
H0 Base agent	70.4 / +6.4 / +5.5 / 0.02	45.2	31.0	38.1
<i>Self-evolving baselines</i>				
A-Evolve (linear chain)	87.1 / +17.6 / +34.1 / 0.29	45.2	47.5	46.3
GEPA-lite	61.5 / +5.3 / +2.0 / -0.02	42.9	28.2	35.5
Meta-Harness-lite	92.2 / +3.1 / +597.8 / 0.06	41.0	29.4	35.2
OctoTools (static)	74.5 / +5.3 / +57.3 / 0.05	38.3	25.6	32.0
<i>Our contributions</i>				
Multi-agent only	86.0 / +38.9 / +39.8 / 0.48	47.5	49.5	48.5
Navigation only	53.1 / +8.7 / -2.8 / -0.04	36.4	37.6	37.0
Multi + Nav	82.1 / +14.2 / +78.4 / 0.32	56.8	50.5	53.7
Multi + Nav + HITL	84.3 / +15.1 / +82.1 / 0.34	57.6	53.4	55.5
<i>Reference</i>				
Expert-designed	85.1 / +19.0 / +95.2 / 0.42	59.3	56.0	57.7

ing by running the solver on every branch and taking the per-task maximum. All three share identical evolver effort; only the routing decision changes. Oracle-branch upper-bounds $V(C_{\Phi}^*(x_t), x_t)$, so (oracle – unconditional) upper-bounds per-task L_{nav} .

Figure 5 reports the three quantitative readings of the design on a common substrate. The top panel binds the static prediction: batches binned by shift magnitude Σ_t (estimated from within-batch variance of oracle-branch identity) plotted against the navigator-minus-unconditional gap; the slope is positive on every benchmark and a paired Wilcoxon signed-rank test on per-task (oracle – unconditional) is significant at $p < 0.001$ in the top shift bins. A falsification control on a temporally shuffled stream (Appendix D.4) collapses the slope to zero, as Proposition 2 predicts when $\Sigma_t \rightarrow 0$. The middle panel reports the dynamic prediction: per-batch accuracy over the CTF-Dojo stream for full-evolution (peaks around batch 12 then degrades, consistent with $dL_{\text{nav}}/dt > 0$ in Proposition 4), early-freeze (plateaus at the peak, having stopped paying the growing navigation loss), and full navigation (monotone, neither degrading nor needing to be frozen). The bottom panel reports per-solve harness size: linear evolution grows from a few kilobytes to roughly 60 KB by end of CTF-Dojo and 176 KB by end of FutureX, while navigation stays bounded because each task loads only one branch’s

workspace — a symptom of L_{nav} accumulation rather than an independent challenge.

Figure 6 grounds the navigation mechanism in a concrete CTF-Dojo run — both the *tree as it grew* (when each branch was forked, on which trigger) and the *routing decision for one task* (what F_{Navigate} reads from each branch and the structured response it emits). The two panels share a run, so the visible branches in the bottom panel are the ones whose creation events are annotated in the top panel.

4.5 Human-in-the-Loop on Novel Task Regimes

Proposition 4 and the orthogonal experience-availability axis of §3 together predict that some task slices cannot be addressed by any evolver acting on $\mathcal{H}_t$ alone, because $\mathcal{H}_t$ does not contain the required signal. To test this we identify the Chinese-language platform-data slice of FutureX *a priori* — English-language Wikipedia and mainstream-news priors do not transfer, and the relevant APIs require credentials the evolver cannot obtain from trajectories alone — and compare the full system with and without the two HITL hooks of §3.5: a *taskboard direction* hook between Analyst and Researcher (semantic guidance for novel regimes) and a *builder credential* hook inside Builder execution (resource gate for authentication-required APIs).

Table 3 shows that HITL lifts zh-finance accuracy by 18.6 points while moving accu-

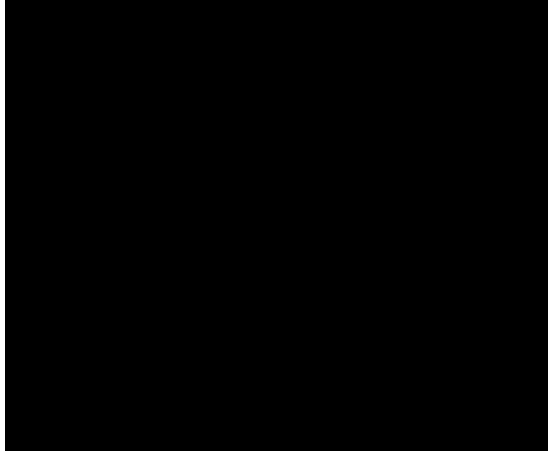


Figure 3: **RQ2 statistics.** Per-ablation artifact distribution and end-of-stream accuracy on CTF-Dojo. *Top:* end-of-stream pass rate per row (A-Evolve 45.2 $\rightarrow$ 49.4 $\rightarrow$ 50.6 $\rightarrow$ 51.2 $\rightarrow$ Full 52.4%). *Bottom:* stacked bars over four artifact categories. Numbers per row, top to bottom: A-Evolve 92/6/2/0; -persistent state 85/10/5/0; -parallel research 70/18/12/0; -role-distinct objectives 62/21/17/0; full multi-agent 35/25/30/10 (prompt / tool / pipeline / distill, %). Only the full configuration populates the multi-file pipeline and cross-cycle distillation tails.

Table 3: HITL impact on the zh-finance slice of FutureX. Gains concentrate on the target slice; performance on other slices is unchanged.

Config	zh-fin.	Other	HITL load
Full system, no HITL	34.6%	52.3%	0 / 4
Full system, + HITL	53.2%	52.4%	4 / 4, $\sim$ 95 s
Δ	+18.6	+0.1	—

racy on the remaining slices by 0.1 points — within measurement noise. The gain is therefore structurally scoped: it appears only where the evolver’s input carries no relevant signal, never as a diffuse “human supervision helps” effect. A typical invocation costs roughly 25 s of human time per cycle (four cycles, about 95 s total), versus zero cost when hooks do not fire.

Figure 7 visualises the targeting pattern: paired bars on the zh-finance slice span an 18.6-point gap, while the same comparison on control slices is within seed noise. Figure 8 shows the two hooks firing in two different cycles of the same FutureX run — they are sequential by construction, since direction must arrive before the Builder has anything to authenticate against.

Taken together, the three contributions are approximately additive. The full system’s gain over A-Evolve decomposes, within measurement noise, into the sum of the multi-agent-only and navigation-only gains, consistent with the orthogonality of L_{evo} and L_{nav} ; HITL contributes an additional increment localized to the experience-insufficient slices. Each component addresses one challenge in isolation, and the composition is what closes the gap to the expert reference.

5 Conclusions

Self-evolving agents deployed on long-horizon, non-stationary, open-ended task streams exhibit three task-intrinsic failure modes that static benchmarks do not surface: non-monotonic evolution under distribution shift, a construction bottleneck at the prompt-only level, and experience insufficiency in novel regimes. Partitioning the agent’s regret into an evolver-class limit $L_{\text{evo}}(\Phi, K)$ and a conditioning limit $L_{\text{nav}}(\varphi)$, with experience availability as a third axis outside the decomposition, yields three composable architectural remedies: a git-backed strategy tree with per-task routing reduces L_{nav} ; a four-phase multi-agent evolver with persistent cross-cycle state expands the constructible harness class and reduces L_{evo} ; and structurally triggered human-in-the-loop hooks supply direction when history is uninformative. Each is independently effective, and together they close most of the gap to a hand-designed expert reference on PolyBench, CTF-Dojo, and FutureX.

Open questions. Branch lifecycle — when to promote a specialised branch back into `main` and when to retire a stale one — remains open, as does co-evolution of the navigator with the strategy tree. Automated detection of experience-insufficiency signals without oracle access would remove the one hand-tuned trigger our HITL layer still relies on. Finally, whether the same three challenges and remedies transfer to domains beyond prediction markets, cybersecurity, and forecasting — robotics, biomedical research, long-form writing — is a direction we leave to future work.

A Full prompts

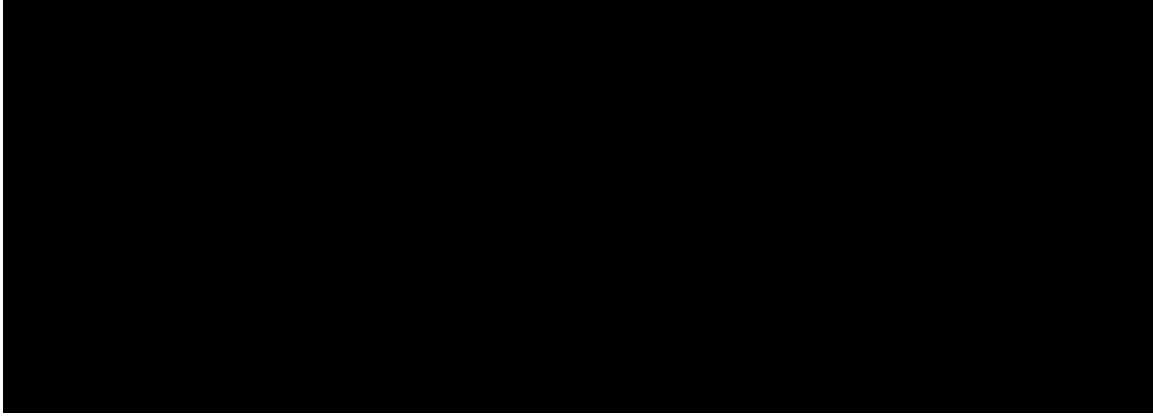


Figure 4: **RQ2 demonstration: a real CTF-Dojo evolution cycle** (provenance: logged cycle, branch/binary-reversing). Five panels left-to-right: (1) Analyst reads failing-batch trajectories and writes a new failure regime entry to `task.board.md`; (2) three parallel Researchers explore disjoint hypotheses in isolation and append `works:true/false` rows to `research_log.jsonl`; (3) Builder reads only the `works:true` records, writes `infra/binary_analysis.py`, and updates `architecture.md`; (4) Verifier executes the new pipeline against held-out tasks and reports pass/fail; (5) the originally-failing task is re-resolved on the next batch using the routed pipeline. Persistent state files (highlighted) link the phases.

740	A.1	F_Navigate system prompt	E.1	A navigation success	763
741	A.2	Analyst, Researcher, Builder,	E.2	A construction success (C2)	764
742		Verifier system prompts	E.3	A HITL success (C3)	765
743	A.3	HITL trigger prompts	E.4	A failure case	766
744	B	Algorithm listings	F	Limitations	767
745	B.1	Algorithm 1: Navigation-aware	Benchmark coverage.	Three benchmarks,	768
746		evolution loop		three domains (prediction markets, cyberse-	769
747	B.2	Algorithm 2: Multi-agent 4-phase		curity, forecasting). The D1/D2/D3 frame's	770
748		cycle		cross-domain generality requires testing on	771
749	C	Benchmark details		domains we have not yet covered (robotics,	772
750	C.1	PolyBench		biomedical, long-form writing).	773
751	C.2	CTF-Dojo	Evolver class is not scalar.	Our L_{evo} ar-	774
752	C.3	FutureX		gument is diagnostic, not predictive. Multi-	775
753	D	Additional tables and figures		agent's empirical advantage is demonstrated	776
754	D.1	Per-batch accuracy curves (all		via ablation and artifact-type analysis (RQ2),	777
755		three benchmarks)		not a formal class-expansion bound.	778
756	D.2	Per-regime slice accuracy	HITL cost model.	We measure HITL cost	779
757	D.3	Artifact-type distribution across		by human time per invocation. In a produc-	780
758		ablations (all benchmarks)		tion deployment, organizational cost (special-	781
759	D.4	Shuffled-order control		ist availability, SLA) may dominate. RQ4 uses	782
760	D.5	Per-branch routing statistics		the author as oracle.	783
761	D.6	Harness size trajectory	Expert reference is author-constructed.		784
762	E	Case studies		The hand-designed harness in Table 2 was	785
				built by the authors with hindsight access to	786
				the task stream. It is an upper bound, not a	787
				competitive SOTA baseline.	788

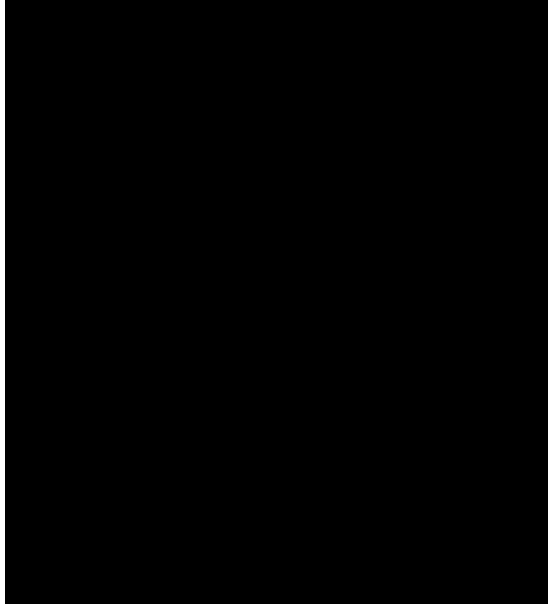


Figure 5: **RQ3 stats.** Three readings on the same evolved tree. *Top*: navigator-minus-unconditional gap versus estimated shift Σ_t , per batch, regression line fit per benchmark (static prediction). *Middle*: per-batch accuracy on the CTF-Dojo stream for unconditional, navigator, and oracle-branch (dynamic prediction). *Bottom*: per-solve harness size in KB — linear evolution grows monotonically; navigation is bounded by the size of the largest single branch.

G Ethics statement

H Reproducibility statement

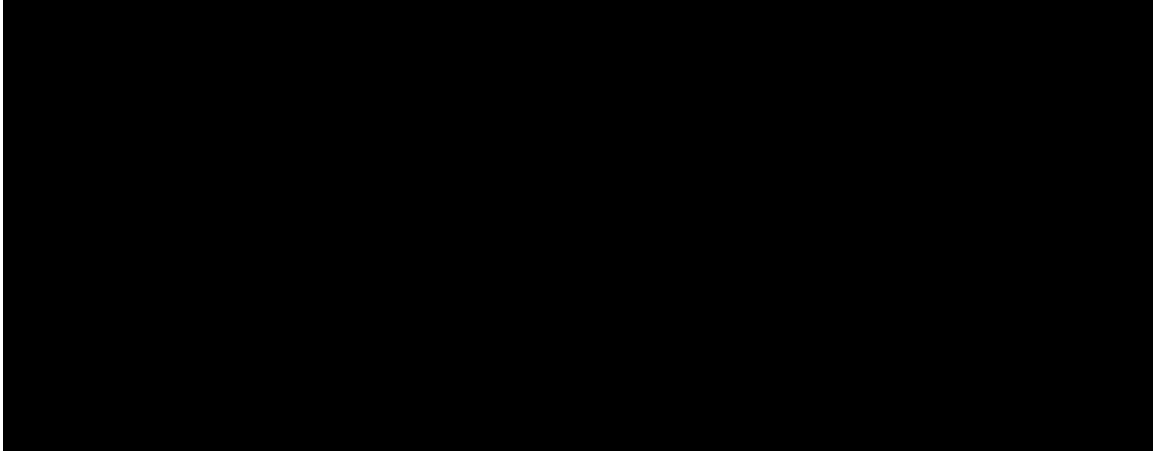


Figure 6: **RQ3 demonstration (combined): tree growth and routing trace** (single CTF-Dojo run). *Top*: strategy tree as it grew over cycles. `main` at $t=0$; `branch/crypto-classical` forked at cycle 2 after the Analyst flagged repeated classical-cipher failures; `branch/binary-reversing` at cycle 4 after stripped-ELF tasks appeared; `branch/web-modern` at cycle 7 after a JWT/SQLi cluster. Each fork is annotated with its trigger. *Bottom*: routing for one task (“RE/2018 — stripped x86-64 ELF, find the flag”). F_{Navigate} reads three side-by-side `git show <branch>:README.md` excerpts and emits `{"branch": "binary-reversing", "confidence": 0.83, "reason": "stripped ELF + RE keywords match this branch's tools/registry; main lacks radare2/Ghidra"}`; the solver then checks out the selected branch and produces the flag.

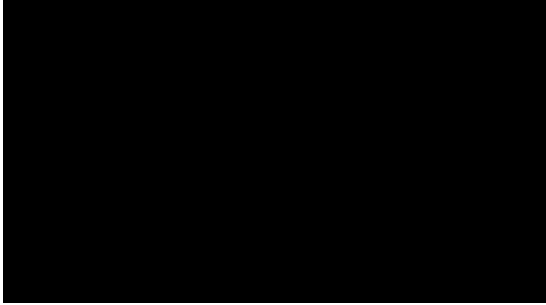


Figure 7: **RQ4 stats**. Targeting pattern of HITL on FutureX: grouped bars over $\{\text{zh-finance, other}\} \times \{\text{no-HITL, +HITL}\}$. The HITL gain is concentrated on the experience-insufficient slice (left pair, +18.6 pts) and is within seed noise on the control slice (right pair, +0.1 pts) — the empirical signature of C3.



Figure 8: **RQ4 demonstration: two HITL hooks across two cycles** (single FutureX run; cycle indices are run-dependent — typically $k=2$ and $k+m=3$). *Cycle k — taskboard direction hook (C3).* The navigator returns `{"branch": null, "confidence": 0.21}` on three zh-finance tasks; Analyst writes a “novel regime, no prior coverage” tag in `task_board.md`. The HITL prompt fires; a one-line direction (“Chinese-language equity prediction; Eastmoney/Sina Finance are canonical sources; use ZH news + macro indicator joins”) is appended and read by the Researchers in the same cycle. *Cycle $k+m$ — builder credential hook.* With direction from cycle k , parallel Researchers identify Eastmoney as a viable source. Builder begins constructing `infra/chinese_finance_pipeline.py`; the first execution returns `403 Forbidden`. The credential hook fires; the human supplies `EASTMONEY_API_KEY`; Builder retries; Verifier confirms coverage on held-out zh-finance tasks. The two interventions are mechanistically distinct (semantic direction vs. resource), fire on independent triggers, and never co-occur in a single cycle.